MINISTERUL EDUCAȚIEI ȘI CERCETĂRII

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

Facultatea de Automatică şi Calculatoare

Suspensie Activă

Documentaţie de proiect — Identificarea Sistemelor

Nume: POP Raul
Specializare: Automatică şi Informatică Aplicată
Grupa: 30135

Introducere

Lucrarea îşi propune să identifice un sistem de ordinul II cu poli complex conjugaţi, având la dispoziţie doar datele experimentale: un vector de intrare şi unul de ieşire. Acestea provin dintr-un model Simulink excitat cu un semnal chirp, adică o sinusoidă a cărei frecvenţă creşte în timp.

Răspunsul din figura de mai jos are aspectul tipic al unui sistem de ordinul doi: la frecvenţe joase ieşirea urmăreşte intrarea, iar pe măsură ce frecvenţa creşte, amplitudinea ei scade.

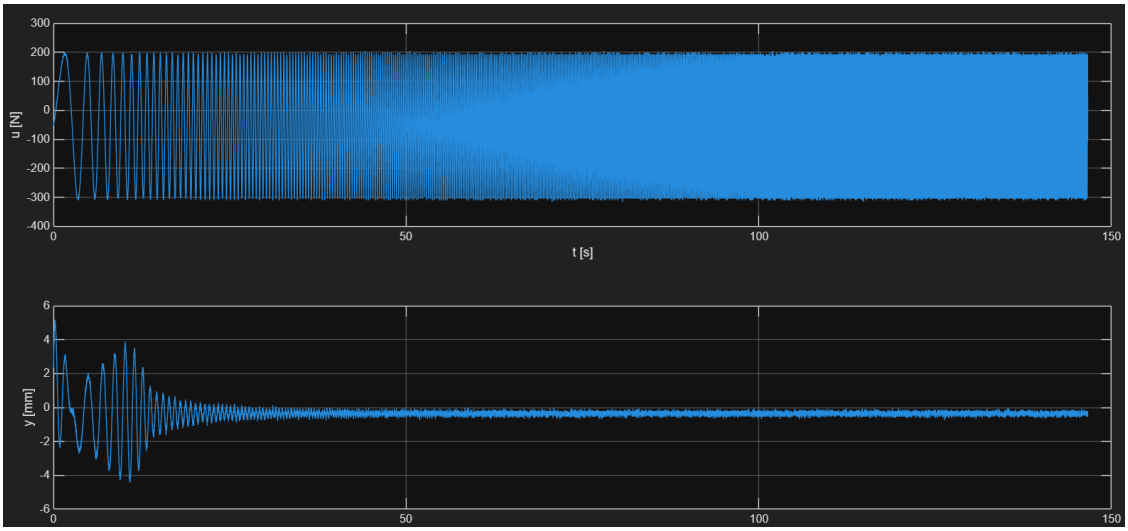


Figura 1 — Semnalele achiziţionate: intrarea u(t) şi ieşirea y(t).

Ca model, alegem funcţia de transfer standard de ordinul doi, cu poli complex conjugaţi:

$$\mathrm{H}(s) = \frac{K{\omega_n}^2}{s^2 + 2\zeta\omega_n s + {\omega_n}^2}$$

Avem de aflat, aşadar, trei parametri: constanta de proporţionalitate K, pulsaţia naturală ωn şi factorul de amortizare ζ. Pentru asta folosim metoda semnalului chirp, descrisă în capitolul următor.

1. Metoda Semnalului Chirp

1.1 Calibrarea semnalului chirp

Pentru ca experimentul să fie util, semnalul chirp trebuie să acopere clar atât frecvenţele joase, cât şi pe cele medii, unde se vede dinamica sistemului şi felul în care sunt atenuate frecvenţele de la intrare.

Din tot experimentul, doar domeniul de frecvenţă [fmin, fmax] mai trebuie calibrat; durata, legea şi viteza de variaţie a frecvenţei, precum şi frecvenţa de eşantionare erau deja stabilite.

Domeniul se alege ca [ωc/10, 10ωc], unde ωc ≈ ωf, ceea ce a condus la valorile:

```
fmin = 0.1/5;
fmax = 10/5;
Ain  = 385;
```

După calibrare, semnalele arată ca în figura următoare:

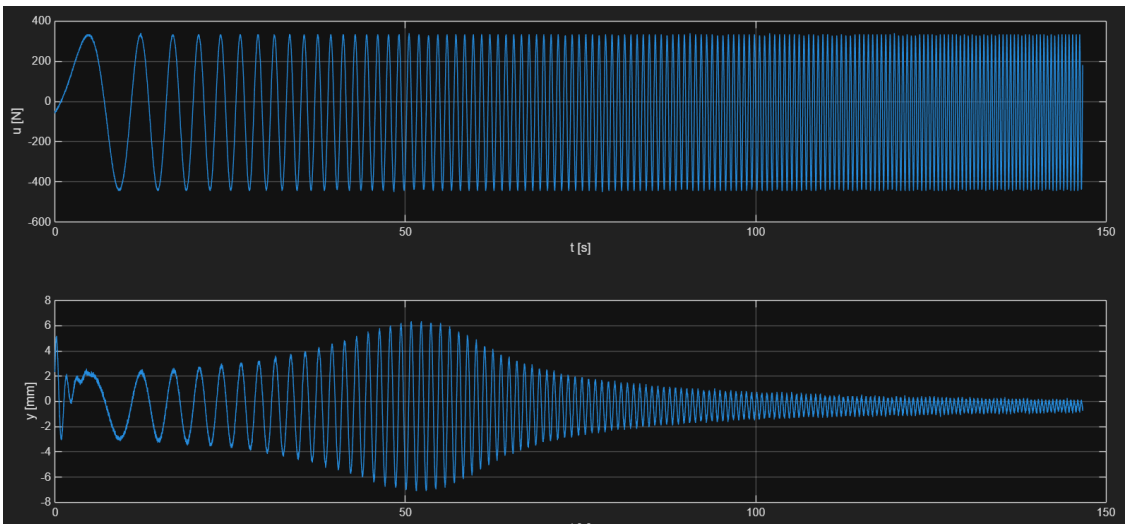


Figura 2 — Semnalele u(t) și y(t) după calibrarea domeniului de frecvență.

1.2 Fundamentare teoretică

Răspunsul sistemului la un chirp se citește ușor pe grafic. La frecvențe mici, ieșirea trece proporțional cu constanta K. În jurul lui ωr amplitudinea crește și formează o „cocoașă", după care scade din nou. Este fenomenul de rezonanță, vizibil și pe diagrama Bode, iar comportamentul de ansamblu este cel al unui filtru trece-jos: semnalul de intrare își schimbă doar frecvența, iar amplitudinea ieșirii se modifică în consecință. Existența rezonanței impune ca ζ să se afle în intervalul (0, √2/2).

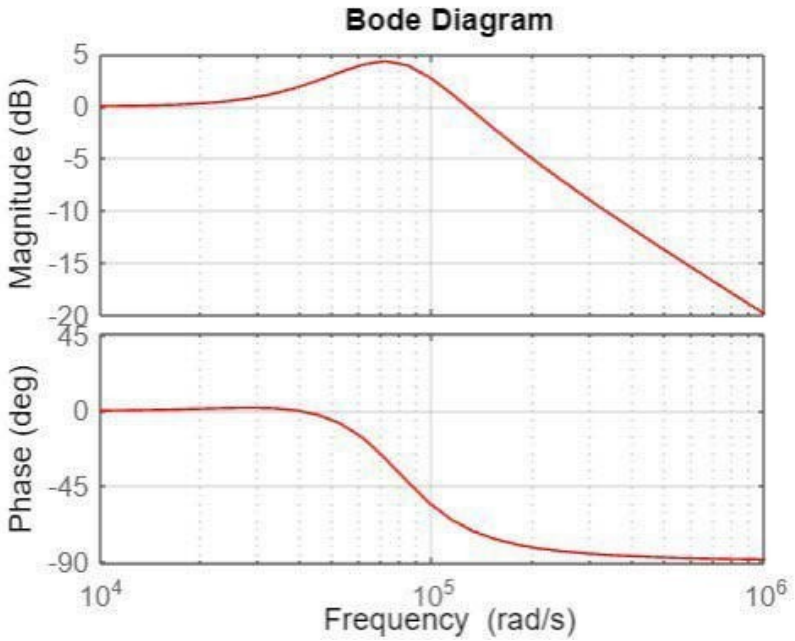


Figura 3 — Diagrama Bode (sursă: r/ControlTheory).

1.3 Determinarea constantei de proporționalitate K

Constanta K se obține din comportamentul la frecvențe joase. Se ia primul maxim al ieșirii, după ce efectul condițiilor inițiale a dispărut, și se raportează la amplitudinea primului maxim al intrării:

$$K \approx \frac{A_y^{\mathrm{CFJ}}}{A_u^{\mathrm{CFJ}}}$$

În cod, aceste maxime sunt indexate de i1 (pentru y) și i2 (pentru u):

```
i1 = 6083; % first clear max for y
i2 = 6178; % first clear max for u

K = y(i1) / u(i2);
```

Alternativ, K s-ar putea calcula ca raport între valorile medii ale semnalelor de intrare și de ieșire în regim staționar.

1.4 Determinarea parametrilor ζ și ωn

Determinarea lui ζ și ωn pornește de la modulul de la rezonanță, adică de la câștigul maxim al ieșirii raportat la cel al intrării:

$$M_r \approx \frac{A_y^r}{A_u^r}$$

Pulsația de rezonanță se obține din perioada de oscilație, calculată ca media dintre perioada din stânga și cea din dreapta vârfului de rezonanță:

$$\omega_{\text{osc}} = \frac{2\pi}{\text{T}_{\text{osc}}}$$

Pe de altă parte, între ωr și parametrii sistemului există relația:

$$\omega_r = \omega_n \sqrt{1 - 2\zeta^2}$$

iar valoarea modulului de la rezonanță, evaluat pe funcția de transfer, este:

$$M_r = |\text{H}(j\omega_r)| = \frac{K}{2\zeta\sqrt{1-\zeta^2}}$$

Din ultima egalitate, prin ridicare la pătrat și rearanjare, rezultă o ecuație bipătrată în ζ:

$$4M_r^2\zeta^4 - 4M_r^2\zeta^2 + K^2 = 0$$

a cărei soluție, aflată în intervalul (0, √2/2), este:

$$\zeta = \sqrt{\frac{1 - \sqrt{1 - \left(\frac{K}{M_r}\right)^2}}{2}}$$

Cum pulsația de rezonanță poate fi aproximată prin cea de oscilație măsurată:

$$\omega_r \approx \omega_{\text{osc}}$$

pulsația naturală se deduce imediat din relația dintre ωr și ωn:

$$\omega_n = \frac{\omega_r}{\sqrt{1 - 2\zeta^2}}$$

Concret, amplitudinile de la rezonanță se calculează din maximul și minimul corespunzătoare ale ieșirii (i3, i4) și ale intrării (i5, i6), iar perioada de oscilație din trei vârfuri succesive din jurul rezonanței (i7, i8, i9):

```
i3 = 25421; % max rez y
i4 = 25782; % min rez y
i5 = 23800; % max u
i6 = 24187; % min u

Amply = (y(i3) - y(i4))/2;
```

```
Amplu = (u(i5) - u(i6))/2;

i7 = 24698; % first peak
i8 = 25383; % rez peak
i9 = 26140; % second peak

ToscPrev  = t(i8) - t(i7);
ToscAfter = t(i9) - t(i8);
Tosc = (ToscPrev + ToscAfter)/2;

wosc = 2*pi/Tosc;% =~ wrez
wr = wosc;
Mr = Amply / Amplu;

zeta = sqrt((1 - sqrt(1 - (K/Mr)^2) )/2);
wn = wr/ sqrt(1 - 2*zeta^2);
```

Rulând codul de mai sus se obţin valorile parametrilor:

$$K = 0.0066, \quad \zeta = 0.2008, \quad \omega_n = 4.5443$$

iar funcţia de transfer identificată, cu valori numerice, devine:

$$\mathrm{H}(s) = \frac{0.1363}{s^2 + 1.825s + 20.65}$$

1.5 Reconstrucţia modelului

Cu parametrii K, ζ şi ωn determinaţi, modelul se reconstruieşte în cele două forme uzuale: funcţia de transfer şi reprezentarea în spaţiul stărilor.

Funcţia de transfer

Funcţia de transfer se construieşte direct din cei trei parametri, iar răspunsul la intrarea măsurată se obţine cu lsim:

```
H = tf(K*wn^2, [1, 2*zeta*wn, wn^2]);
y_sim_tf = lsim(H, u, t);
```

Simularea porneşte implicit din condiţii iniţiale nule, aşa că răspunsul estimat are nevoie de un timp până să ajungă la cel real. De aceea erorile J şi eps_MPN ies mai mari decât la reprezentarea în spaţiul stărilor (secţiunea 1.6).

Spaţiul stărilor

Aceleaşi trei valori definesc şi modelul în spaţiul stărilor. Matricea de stare este:

$$\mathrm{A} = \begin{bmatrix} 0 & 1 \\ -\omega_n{}^2 & -2\zeta\omega_n \end{bmatrix};$$

vectorul de intrare:

$$\mathrm{B} = \begin{bmatrix} 0 \\ K\omega_n{}^2 \end{bmatrix};$$

iar ieşirea este prima stare, fără legătură directă cu intrarea:

$$\mathrm{C} = \begin{bmatrix} 1 & 0 \end{bmatrix}; \mathrm{D} = 0;$$

Cu valorile identificate, matricele A şi B devin:

$$\mathrm{A} = \begin{bmatrix} 0 & 1 \\ -20.65 & -1.825 \end{bmatrix}$$

$$\mathrm{B} = \begin{bmatrix} 0 \\ 0.1363 \end{bmatrix}$$

Modelul se simulează tot cu lsim, de data aceasta cu condiții inițiale explicite:

```
sys = ss(A, B, C, D);
y_sim_ss = lsim(sys, u, t, [y(1) 25]);
```

Poziția inițială este cunoscută din semnalul măsurat. Viteza inițială se poate afla fie analitic, derivând semnalul și luând prima valoare, fie numeric, prin metoda înjumătățirii intervalului (bisecție), aleasă aici. Prin calibrări succesive s-a ajuns la valoarea 25, dincolo de care orice altă modificare înrăutățea eroarea de validare.

1.6 Validarea modelului

Modelele se compară cu semnalul măsurat prin doi indicatori, cei prezentați la cursul de Identificarea Sistemelor. Primul este:

$$J = \frac{\| \varepsilon \|_2}{\sqrt{N}}$$

iar al doilea, normalizat în raport cu variația ieșirii în jurul mediei:

$$\varepsilon_{\mathrm{MPN}} = \frac{\| \varepsilon \|_2}{\| y - \bar{y} \|_2}$$

Pentru a reduce și mai mult eroarea, ieșirea măsurată a fost filtrată în două moduri: cu un filtru Butterworth de ordinul 2 și cu un filtru de tip moving average.

Modelul în spațiul stărilor

Modelul în spațiul stărilor se suprapune foarte bine peste semnalul măsurat, atât brut, cât și filtrat. Filtrarea Butterworth duce eroarea la eps_MPN = 3,20%, iar moving average la eps_MPN = 2,82%, ambele sub pragul de 10% impus.

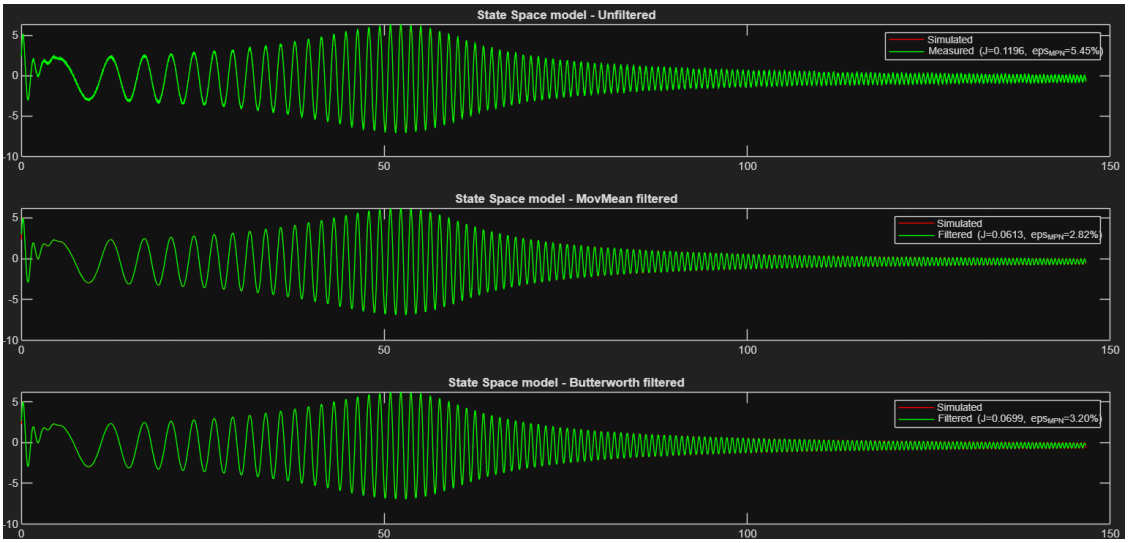


Figura 4 — Validarea modelului în spațiul stărilor.

Modelul funcţie de transfer

Modelul funcție de transfer are o eroare mult mai mare, eps_MPN ≈ 14%, deşi foloseşte exact aceiaşi parametri K, ζ şi ωn. Diferența vine din condițiile inițiale: în spațiul stărilor ele au fost date explicit ([y(1) 25]), pe când lsim aplicat funcției de transfer pornește din zero. Că motivul este acesta, şi nu o alegere nefericită a porțiunii de date, se vede din faptul că eroarea mare nu se mai menține pe intervale de testare neutilizate la calibrare.

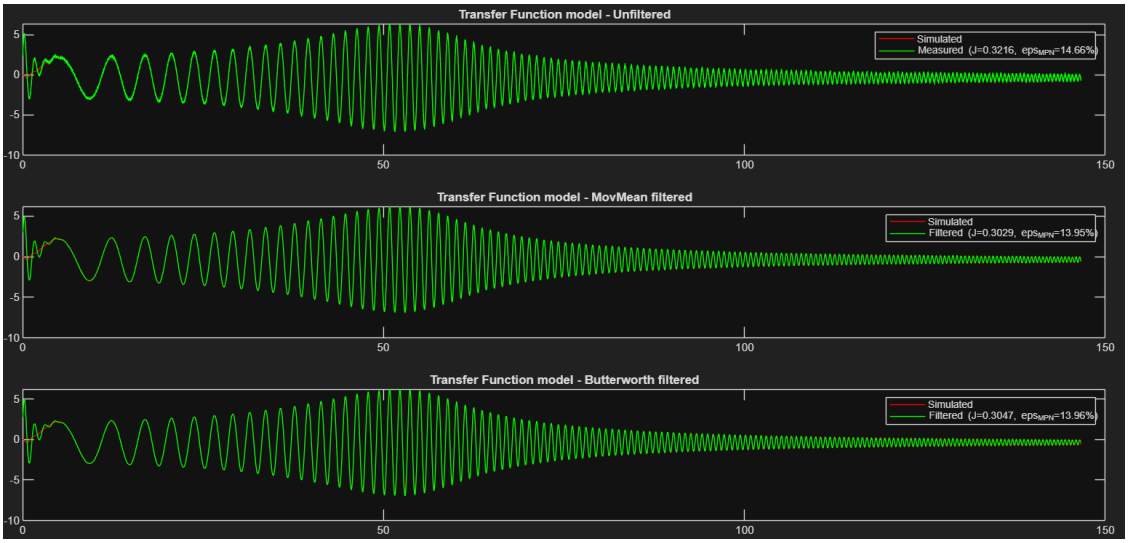


Figura 5 — Validarea modelului funcție de transfer.

Inițial, eroarea semnalului era de 11%, redusă ulterior prin ajustarea fină a indicilor. Lipsa unei viteze inițiale bune lansa funcția mult deasupra celei inițiale, iar un ζ ales prea mare diminua amplitudinea în zona de rezonanță şi o amplifica în afara ei, ceea ce explica eroarea mare. A fost nevoie de ajustări şi la K, deoarece o singură valoare afectată de zgomot îi modifica puternic valoarea.

Anexă — Cod sursă MATLAB

În anexă este redat întregul cod MATLAB folosit în această metodă: calculul constantei K, determinarea lui ζ şi ωn prin relaţiile de mai sus, reconstrucţia modelului (funcţie de transfer şi spaţiul stărilor), filtrarea ieşirii şi calculul indicatorilor de performanţă.

```
%% System identification
% Cur   s 5 - Modalitatea de calibrare a intrarii
% Curs 4 - Metodologie

i1 = 6083; % first clear max for y
i2 = 6178; % first clear max for u

K = y(i1) / u(i2);

i3 = 25421; % max rez y
i4 = 25782; % min rez y

i5 = 23800; % max u
i6 = 24187; % min u

Amply = (y(i3) - y(i4))/2;
Amplu = (u(i5) - u(i6))/2;
% 3 pct de maxim in jurul rezonantei => wk-1 < wr < wk

i7 = 24698; % first peak
i8 = 25383; % rez peak
i9 = 26140; % second peak

ToscPrev  = t(i8) - t(i7);
ToscAfter = t(i9) - t(i8);
Tosc = (ToscPrev + ToscAfter)/2;

wosc = 2*pi/Tosc;% =~ wrez
wr = wosc;
Mr = Amply / Amplu;


zeta = sqrt((1 - sqrt(1 - (K/Mr)^2) )/2);
wn = wr/ sqrt(1 - 2*zeta^2);

%% Transform
H = tf(K*wn^2, [1 2*zeta*wn wn^2]);
y_sim_tf = lsim(H,u,t);

%% State Space
A = [ 0 1;
   -wn.^2 -2*zeta*wn];
B = [0;
   K*wn.^2];
C = [1 0];
D = 0;

sys = ss(A,B,C,D);
y_sim_ss = lsim(sys,u,t, [y(1) 25]);

%% Filtering
idx_val = (i9+1):(length(t)- 1000); % Unused values

% Moving average filter
windowSize = 55;
y_mov = movmean(y,windowSize);

% Butterworth filter
fs = 1/Ts;
fc = 3;
ord = 2;
```

```
[b_filt, a_filt] = butter(ord, fc/(fs/2), 'low');
y_butter = filtfilt(b_filt, a_filt, y);
%% Performance indexes
% J = ||eps||_2 / sqrt(N)
% eps_MPN = ||eps||_2 / ||y - mean(y)||_2

% State Space - nefiltrat
J_ss_full    = norm(y_sim_ss - y) / sqrt(length(y));
eMPN_ss_full = norm(y_sim_ss - y) / norm(y - mean(y));
J_ss_val     = norm(y_sim_ss(idx_val) - y(idx_val)) / sqrt(length(idx_val));
eMPN_ss_val  = norm(y_sim_ss(idx_val) - y(idx_val)) / norm(y(idx_val) - mean(y(idx_val)));

% State Space - movmean
J_ss_mov_full    = norm(y_sim_ss - y_mov) / sqrt(length(y_mov));
eMPN_ss_mov_full = norm(y_sim_ss - y_mov) / norm(y_mov - mean(y_mov));
J_ss_mov_val     = norm(y_sim_ss(idx_val) - y_mov(idx_val)) / sqrt(length(idx_val));
eMPN_ss_mov_val  = norm(y_sim_ss(idx_val) - y_mov(idx_val)) / norm(y_mov(idx_val) - mean(y_mov(idx_val)));

% State Space - Butterworth
J_ss_butter_full    = norm(y_sim_ss - y_butter) / sqrt(length(y_butter));
eMPN_ss_butter_full = norm(y_sim_ss - y_butter) / norm(y_butter - mean(y_butter));
J_ss_butter_val     = norm(y_sim_ss(idx_val) - y_butter(idx_val)) / sqrt(length(idx_val));
eMPN_ss_butter_val  = norm(y_sim_ss(idx_val) - y_butter(idx_val)) / norm(y_butter(idx_val) - mean(y_butter(idx_val)));

% Transfer Function - nefiltrat
J_tf_full    = norm(y_sim_tf - y) / sqrt(length(y));
eMPN_tf_full = norm(y_sim_tf - y) / norm(y - mean(y));
J_tf_val     = norm(y_sim_tf(idx_val) - y(idx_val)) / sqrt(length(idx_val));
eMPN_tf_val  = norm(y_sim_tf(idx_val) - y(idx_val)) / norm(y(idx_val) - mean(y(idx_val)));

% Transfer Function - movmean
J_tf_mov_full    = norm(y_sim_tf - y_mov) / sqrt(length(y_mov));
eMPN_tf_mov_full = norm(y_sim_tf - y_mov) / norm(y_mov - mean(y_mov));
J_tf_mov_val     = norm(y_sim_tf(idx_val) - y_mov(idx_val)) / sqrt(length(idx_val));
eMPN_tf_mov_val  = norm(y_sim_tf(idx_val) - y_mov(idx_val)) / norm(y_mov(idx_val) - mean(y_mov(idx_val)));

% Transfer Function - Butterworth
J_tf_butter_full    = norm(y_sim_tf - y_butter) / sqrt(length(y_butter));
eMPN_tf_butter_full = norm(y_sim_tf - y_butter) / norm(y_butter - mean(y_butter));
J_tf_butter_val     = norm(y_sim_tf(idx_val) - y_butter(idx_val)) / sqrt(length(idx_val));
eMPN_tf_butter_val  = norm(y_sim_tf(idx_val) - y_butter(idx_val)) / norm(y_butter(idx_val) - mean(y_butter(idx_val)));

%% Plots - State Space model
figure
subplot(3,1,1)
plot(t,y_sim_ss,'r', t, y, 'g');
title('State Space model - Unfiltered')
legend('Simulated', sprintf('Measured (J=%.4f, eps_{MPN}=%.2f%%)', J_ss_full, 100*eMPN_ss_full))

subplot(3,1,2)
plot(t,y_sim_ss,'r', t, y_mov, 'g');
title('State Space model - MovMean filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_ss_mov_full, 100*eMPN_ss_mov_full))

subplot(3,1,3)
plot(t,y_sim_ss,'r', t, y_butter, 'g');
title('State Space model - Butterworth filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_ss_butter_full, 100*eMPN_ss_butter_full))

%% Plots - Transfer Function model
figure
subplot(3,1,1)
plot(t,y_sim_tf,'r', t, y, 'g');
title('Transfer Function model - Unfiltered')
legend('Simulated', sprintf('Measured (J=%.4f, eps_{MPN}=%.2f%%)', J_tf_full, 100*eMPN_tf_full))

subplot(3,1,2)
plot(t,y_sim_tf,'r', t, y_mov, 'g');
title('Transfer Function model - MovMean filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_tf_mov_full, 100*eMPN_tf_mov_full))
```

```
subplot(3,1,3)
plot(t,y_sim_tf,'r', t, y_butter, 'g');
title('Transfer Function model - Butterworth filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_tf_butter_full, 100*eMPN_tf_butter_full))
```